0494. Target Sum

m | Category: 2-D Dynamic Programming | LeetCode: https://leetcode.com/probl

Problem

- Put + or - before each number. Count the assignments whose total equals target.

Intuition

- Let P be the numbers with +. Then sum(P) - (total - sum(P)) = target, so sum(P) = (total + target) / 2.
- The problem becomes "count subsets with a given sum": 0/1 knapsack counting.

Brute Force

- Try all 2^n sign assignments (or memoise on (index, running sum)).

Optimal Approach

- If |target| > total or (total + target) is odd, return 0.
- S = (total + target) / 2; ways[0] = 1.
- For each x: for t from s down to x: ways[t] += ways[t - x].

Worked Example

```
nums [1,1,1,1,1], target 3.

total=5 -> s=(5+3)/2=4
count subsets of five 1s with sum 4 = C(5,4) = 5
```

Java Solution

```java
public class TargetSum {
    public int findTargetSumWays(int[] nums, int target) {
        int total = 0;
        for (int x : nums) total += x;
        if (Math.abs(target) > total || (total + target) % 2 != 0) return 0;
        int s = (total + target) / 2;
        int[] ways = new int[s + 1];
        ways[0] = 1;
        for (int x : nums)
            for (int t = s; t >= x; t--) ways[t] += ways[t - x];
        return ways[s];
    }
}
```

Complexity

- Time: O(n s). Space: O(s).

Pitfalls

- Zeros double the count (+0 and -0); the DP handles it because x = 0 adds ways[t] to itself once per zero.